

Exercício 1: O que a função `fatorDeBalancamento(NULL)` retorna e qual verificação interna impede um erro de segmentação (segmentation fault)?

R: A verificação que evita o segmentation fault é tipo:

```
if(no == NULL)
    return 0;
```

Isso impede o acesso a campos inválidos como `no->esq` ou `no->dir`.

Exercício 2: O nó folha 1 (com `esq == NULL` e `dir == NULL`) possui altura = 0. Calcule o retorno de `fatorDeBalancamento(1)`.

R: O nó folha 1 possui:

`altura(esq) = -1, altura(dir) = -1`

$FB = (-1) - (-1) = 0$, $FB = (-1) - (-1) = 0$

`fatorDeBalancamento(1) = 0`

Exercício 3: Antes da rotação, o nó 5 possui o filho esquerdo 2 (altura 1) e o filho direito nulo (altura -1). Qual o valor de `fatorDeBalancamento(5)` e o que ele sinaliza?

R: Para o nó 5:

filho esquerdo altura = 1, filho direito altura = -1

$FB = 1 - (-1) = 2$, $FB = 1 - (-1) = 2$

Isso sinaliza que a árvore está desbalanceada para a esquerda e precisa de rotação.

Exercício 4: Logo após a execução das duas atribuições iniciais de `rotacaoDireita`, quais nós específicos da árvore `5 -> 2 -> 1` estão armazenados nas variáveis locais `novaRaiz` e `filhoBackup`?

R: Na árvore:

```
    5
   /
  2
 /
1
```

Após:

`novaRaiz = raiz->esq;`

`filhoBackup = novaRaiz->dir;`

Temos:

`novaRaiz` → nó 2

`filhoBackup` → NULL (porque o nó 2 não possui filho direito)

Exercício 5: Durante a execução de rotacaoDireita, qual nó passa a ser o novo filho esquerdo de raiz (nó 5) através da atribuição `raiz->esq = filhoBackup`; e por que essa linha é necessária?

R: O novo filho esquerdo de raiz (nó 5) será o nó armazenado em `filhoBackup`.

Nesse caso específico, será NULL.

A linha:

`raiz->esq = filhoBackup`;

é necessária para preservar a subárvore direita de `novaRaiz` durante a rotação. Sem ela, parte da árvore poderia ser perdida.

Exercício 6: Ao término da função `rotacaoDireita`, qual ponteiro de nó é retornado e qual o seu papel geométrico final na árvore rebalanceada?

R: A função retorna o ponteiro `novaRaiz`.

]

Após a rotação, esse nó se torna a nova raiz geométrica da subárvore rebalanceada.

Na árvore dada, o nó 2 passa a ocupar o topo:

```
  2
 / \
1   5
```

Exercício 7: Por que a linha que atualiza `raiz->altura` é executada estritamente antes da linha que atualiza `novaRaiz->altura`?

R: Porque a altura de `novaRaiz` depende da altura atualizada de `raiz`.

Depois da rotação, `raiz` vira filho de `novaRaiz`. Portanto, primeiro calcula-se corretamente a altura do filho (`raiz`) e só depois a do pai (`novaRaiz`).

Exercício 8: Na árvore `5 -> 2 -> 1`, após as trocas de ponteiros, qual será a nova altura calculada e armazenada em `raiz->altura` (nó 5)?

R: Após a rotação:

```
  2
 / \
1   5
```

O nó 5 fica sem filhos:

`altura(esq) = -1`

`altura(dir) = -1`

Então:

`altura(5) = max(-1, -1) + 1 = 0`

$altura(5) = \max(-1, -1) + 1 = 0$

Logo, raiz $\rightarrow$ altura (nó 5) será 0.

Exercício 9: Se a linha `filhoBackup = novaRaiz->dir;` fosse deletada do código, o que aconteceria com a estrutura de dados após a atribuição `novaRaiz->dir = raiz;`?

R: Se a linha:

`filhoBackup = novaRaiz->dir;`

fosse removida, a subárvore direita de `novaRaiz` seria perdida quando fosse executado:

`novaRaiz->dir = raiz;`

Ou seja, ocorreria perda de referências na estrutura da árvore.

Exercício 10: Uma rotação direita pura ocorre quando raiz (nó 5) tem fator +2 e `novaRaiz` (nó 2) tem fator +1. Qual será o novo fator de balanceamento desses dois nós imediatamente após o encerramento de `rotacaoDireita`?

R: Antes da rotação:

nó 5 $\rightarrow$ fator +2

nó 2 $\rightarrow$ fator +1

Depois da rotação direita simples, ambos ficam balanceados:

$FB(5) = 0$

$FB(2) = 0$

$FB(5) = 0; FB(2) = 0$